

Formal Verification (CS-550)

Viktor Kuncak, EPFL

<https://lara.epfl.ch/w/fv>

Formal Verification

Goal: rigorously prove that computer systems “do what they should do”

“do what they should do” = satisfy a specification

How?

1. Define a mathematically rigorous notion of a system satisfying a specification
2. Use combination of automated tools and human effort to construct the proof

We will learn: how to **use** and **build** tools for computer-aided formal verification

Where “Formal” Comes From

form - how we write things down, in contrast to meaning (what they are)

Example: $f : \mathbb{R} \rightarrow \mathbb{R}$ where $f(x) = x^2$ describes a function using some finite sequence of symbols. It denotes some infinite object that maps all possible real numbers to their squares.

Formalism: proposed by Hilbert advanced by Russell and Whitehead and others, suggests that we can *in principle* do mathematics by identifying the precise rules for manipulating symbols and following them. It was proposed at the time when doing so explicitly would be a tedious mathematical activity and was mainly aimed at the goal of using mathematics to analyze the foundation of mathematics itself (metamathematics, which led to formal logic and model theory).

With computers, we find that we can do this not only *in principle*, but also *in practice* and that it covers every kind of mathematics we do. It took until the appearance of Lean for more mathematicians to use formal methods.

Formal Methods for Computer Systems

Computer science benefits from the same foundational mechanisms to check rigorous proofs as mathematics.

Differences:

- ▶ mathematics (and theoretical computer science) focuses on widely reusable results and, when possible, elegant theorems and definitions, $x^{n+2} + y^{n+2} \neq z^{n+2}$.
- ▶ computer science systems need to prove things about large and messy artifacts.
e.g. *Theorem: These 100K lines of code have no buffer overflow and behave from outside like this 10k simpler (less efficient) prototype.*

Because computer system descriptions are large and theorems not very deep, there is more emphasis on algorithms to automatically construct proofs (decision procedures, model checking, abstract interpretation) that complete certain specific kinds of proofs. This may be somewhat changing with LLMs, with proof checking becoming more important for computer systems. We will learn how to construct systems to check and find proofs.

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work.

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work. This is hard work.

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work. This is hard work.

Why bother? Maybe we can use a **really fast** fuzz tester?

Suppose we want to **test** that addition of two **Long** integer values is commutative by trying all possible values.

```
assert(x + y == y + x)
```

(The values x and y are arbitrary, they may come, e.g., from input.)

Suppose we can run 10 tests every **nanosecond**. How long to test **all** cases?

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work. This is hard work.

Why bother? Maybe we can use a **really fast** fuzz tester?

Suppose we want to **test** that addition of two **Long** integer values is commutative by trying all possible values.

```
assert(x + y == y + x)
```

(The values x and y are arbitrary, they may come, e.g., from input.)

Suppose we can run 10 tests every **nanosecond**. How long to test **all** cases?

Number of tests: $2^{64} \cdot 2^{64} = 2^{128} > 10^{38}$

Seconds: 10^{28}

Days: $1.15 \cdot 10^{23}$

Years: $3.15 \cdot 10^{20}$

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work. This is hard work.

Why bother? Maybe we can use a **really fast** fuzz tester?

Suppose we want to **test** that addition of two **Long** integer values is commutative by trying all possible values.

```
assert(x + y == y + x)
```

(The values x and y are arbitrary, they may come, e.g., from input.)

Suppose we can run 10 tests every **nanosecond**. How long to test **all** cases?

Number of tests: $2^{64} \cdot 2^{64} = 2^{128} > 10^{38}$

Seconds: 10^{28}

Days: $1.15 \cdot 10^{23}$

Years: $3.15 \cdot 10^{20}$

Ten billion times since “big bang”.

Comparison to Testing

We test computer systems and we should. But in formal verification, we go beyond: make mathematical models and prove that the systems work. This is hard work.

Why bother? Maybe we can use a **really fast** fuzz tester?

Suppose we want to **test** that addition of two **Long** integer values is commutative by trying all possible values.

```
assert(x + y == y + x)
```

(The values x and y are arbitrary, they may come, e.g., from input.)

Suppose we can run 10 tests every **nanosecond**. How long to test **all** cases?

Number of tests: $2^{64} \cdot 2^{64} = 2^{128} > 10^{38}$

Seconds: 10^{28}

Days: $1.15 \cdot 10^{23}$

Years: $3.15 \cdot 10^{20}$

Ten billion times since “big bang”. Don’t even ask about $x + (y + z) == (x + y) + z$

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- ▶ it's a mathematical **theorem** about integers modulo 2^{64}
- ▶ we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also uses **logical rules**, studied in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$.

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- ▶ it's a mathematical **theorem** about integers modulo 2^{64}
- ▶ we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also uses **logical rules**, studied in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$. Verifiers also use **automated theorem proving procedures**, which can discover an infinite number of facts using known theorems and rules.

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- ▶ it's a mathematical **theorem** about integers modulo 2^{64}
- ▶ we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also uses **logical rules**, studied in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$. Verifiers also use **automated theorem proving procedures**, which can discover an infinite number of facts using known theorems and rules.

Different kinds of proving procedures, depending on success guarantees:

- ▶ decision procedures (linear arithmetic, SAT, bitvectors), proof checkers: algorithms with upper bounds on time by which they terminate with yes/no answer

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- ▶ it's a mathematical **theorem** about integers modulo 2^{64}
- ▶ we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also uses **logical rules**, studied in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$. Verifiers also use **automated theorem proving procedures**, which can discover an infinite number of facts using known theorems and rules.

Different kinds of proving procedures, depending on success guarantees:

- ▶ decision procedures (linear arithmetic, SAT, bitvectors), proof checkers: algorithms with upper bounds on time by which they terminate with yes/no answer
- ▶ semi-decision procedures (first-order logic provers): no upper bound, but for every theorem there exists a time by which they discover that it holds (complete)

Feasible Alternative: Automated Theorem Proving

A modern software verifier has **built-in** knowledge of commutativity.

- ▶ it's a mathematical **theorem** about integers modulo 2^{64}
- ▶ we can also prove theorems about unbounded integers (need infinitely many tests)

A verifier also uses **logical rules**, studied in **formal (mathematical) logic**, to take existing theorems, like $x + y = y + x$, and derive new ones, like $x + (a + 1) = (a + 1) + x$. Verifiers also use **automated theorem proving procedures**, which can discover an infinite number of facts using known theorems and rules.

Different kinds of proving procedures, depending on success guarantees:

- ▶ decision procedures (linear arithmetic, SAT, bitvectors), proof checkers: algorithms with upper bounds on time by which they terminate with yes/no answer
- ▶ semi-decision procedures (first-order logic provers): no upper bound, but for every theorem there exists a time by which they discover that it holds (complete)
- ▶ heuristics (simplifiers, induction heuristics, type inference): even if formula is valid, they may run forever, or say “do not know”

Compiling to Formulas (example: Stainless verifier)

How do we go from program correctness statements to theorems?

Compiling to Formulas (example: Stainless verifier)

How do we go from program correctness statements to theorems?

Compile programs and properties to formulas!

We call these formulas **verification conditions**.

If verification condition is valid formula, then program satisfies specification.

Analogy:

programming language compiler	verification-condition generator
program $\rightarrow$ machine code	(program + specification) $\rightarrow$ formula

Compiling to Formulas (example: Stainless verifier)

How do we go from program correctness statements to theorems?

Compile programs and properties to formulas!

We call these formulas **verification conditions**.

If verification condition is valid formula, then program satisfies specification.

Analogy:

programming language compiler	verification-condition generator
program $\rightarrow$ machine code	(program + specification) $\rightarrow$ formula

$$\boxed{\text{program verifier}} = \boxed{\text{verification condition generator}} \oplus \boxed{\text{theorem prover}}$$